Large Language Models Adaptation for Web Applications Attacks Detection

Sidwendluian Romaric Nana
Laboratoire de Mathématiques et d'Informatique (LAMI)
Université Joseph KI-ZERBO
Ouagadougou, Burkina Faso
sidnanaroma@gmail.com

Didier Bassolé
Laboratoire de Mathématiques et d'Informatique (LAMI)
Université Joseph KI-ZERBO
Ouagadougou, Burkina Faso
dbassole@gmail.com

Désiré Guel
Laboratoire de Mathématiques et d'Informatique (LAMI)
Université Joseph KI-ZERBO
Ouagadougou, Burkina Faso
guel.desire@gmail.com

Oumarou Sié
Laboratoire de Mathématiques et d'Informatique (LAMI)
Université Joseph KI-ZERBO
Ouagadougou, Burkina Faso
oumarou.sie@gmail.com

2025 Conference on Information Communications Technology and Society (ICTAS) | 979-8-3315-3155-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICTAS64866.2025.11155839

***Abstract*—Large Language Models (LLMs) represent a major advance in the field of deep learning. Their ability to understand long-term dependencies between words in a sentence has completely revolutionized natural language processing. Based on the architecture of transformers, LLMs are trained to solve common linguistic problems, such as text classification, question answering, text generation, document summarization, etc. LLMs can be pre-trained and then fine-tuned to perform tasks in specific domains. In the field of cybersecurity, we have proposed in our previous work, a LLM-based approach for web applications vulnerabilities detection. In this paper, we implement our approach, which uses adaptation techniques such as fine-tuning to detect and classify common web attacks like XSS, Directory Traversal, SQL injection, Command Injection and benign payloads recognition. LLMs used for fine-tuning are : BERT (encoder model), Llama (decoder model), Flan-T5 (encoder-decoder model). This is one of the first comparative evaluations of BERT, Llama, and Flan-T5 fine-tuned on real-world web attack payloads. We used several fine-tuning techniques including Quantization, Low-Rank Adaptation, Supervised Fine-Tuning or freezing the weights of the LLMs layers and gradually adding fully connected layers. For each of the LLMs, we obtained an accuracy rate of over 98% after fine-tuning. Since we were working with unbalanced dataset, we also evaluated models using metrics such as Balanced Accuracy, Geometric Mean, Matthews Correlation Coefficient, Fowlkes-Mallows Index. The fine-tuned BERT and Flan-T5 models proved to be more robust to unbalanced data. Evaluation with the confusion matrix shows that fine-tuned models correctly classify web application attacks. This confirms that fine-tuning basic models is effective for specific cybersecurity tasks.**

***Index Terms*—Fine-tuning, Large Language Model, Web application attacks, Detection**

I. Introduction

Web attacks have been on the increase in recent years, and are becoming more and more complex. These attacks affect all sectors of activity (education, government, healthcare, finance, insurance, communications, etc.). The second quarter of 2024 saw a 30% increase in cyber attacks compared with the second quarter of 2023, the biggest rise in the last two years [1]. Traditional detection tools of cyber attacks are often ineffective in the face of the dynamic and evolving nature of the threats associated with malicious URLs. It is therefore necessary to adapt tools to new trends.

Research on vulnerability detection in web applications using Deep Learning (DL) has become increasingly active since 2019 [2]. Prior works have focused more on binary classification, but this does not reflect real-world environment. Additionally, model performance evaluation relies on classical metrics which are not adapted for unbalanced data. In previous work [3], an approach using LLMs to detect vulnerabilities in web applications has been proposed. This approach takes advantage of the emergence of LLMs, their ability to work on textual data and the possibility of adapting them to solve specific tasks in other domains with fine-tuning.

In this study, we focus on the detection and classification of web vulnerabilities : Cross-site scripting (XSS), Command Injection (CMDi), Directory Traversal (DT), SQL Injection (SQLi) and benign payloads (normal traffic).

To do this, we are implementing fine-tuning strategies on three (03) LLMs (BERT, Llama, Flan-T5) to adapt them to the detection of web attacks using the propagation of malicious URLs. Since payloads are strings of characters, we take advantage of the power of LLMs for processing textual data.

The rest of this paper is organized as follows. Section II deals with related works. In this section, we review usage of LLMs in cybersecurity domain, present different types of architecture of LLMs and adaptation techniques. In section III, we present our dataset and fine-tuning strategies of LLMs to detect web applications attacks. Results are presented in section IV. We conclude this work in section V.

II. Related Works

A. LLMs in Cybersecurity

Deng et al. [4] have proposed PentestGPT, a tool for automating penetration tests based on GPT. PentestGPT is able to detect security vulnerabilities and propose solutions to

Authorized licensed use limited to: Nokia. Downloaded on September 18,2025 at 07:23:36 UTC from IEEE Xplore. Restrictions apply.

correct them. This tool significantly improves the accuracy rate compared with conventional penetration testing approaches.

By exploiting the Natural Language Processing (NLP) capabilities of LLMs, Happe et al. [5] have conducted experiments to detect software vulnerabilities and perform privilege escalation on vulnerable machines.

Hassanin and Moustafa [6] present a literature review on the use of LLMs in cybersecurity covering several aspects such as cyber threat intelligence, vulnerability assessment of digital assets, computer network security, privacy preservation, etc. Challenges remain and need to be addressed in order to improve the performance of LLMs in the field of cybersecurity and facilitate their integration into real environments such as adapting LLMs to the cybersecurity domain.

Cao et al. [7] present RealVul, a framework for detecting PHP code vulnerabilities based on LLMs. Using normalization techniques, authors detect pieces of code that are likely to trigger vulnerabilities (XSS, SQLi). These 'candidate' codes are pre-processed, then used to fine-tune LLMs (CodeLama-7b, StarCoder2-7b, StarCoder2-3b, CodeT5p-770m and CodeT5-base). Analysis of the results shows that the performance of the fine-tuned models has improved, with a F1-score significantly higher than that of the basic models.

In a recent study, Le et al. [8] proposed a multi-label binary classification framework for detecting SQLi, CMDi, and XSS vulnerabilities using fine-tuned models (DistilBERT, ALBERT, and ArmoRM-Llama3-8B). The fine-tuning strategy used was to freeze the weights of the base models and add a fully connected layer for classification. During fine-tuning, only the classification layer is trained. In this study, ArmoRM-Llama3-8B performed slightly better than the other models. Unfortunately, since the datasets used are not described (balanced or unbalanced), it is not possible to evaluate the choice of metrics used. Furthermore, DistilBERT, ALBERT, and ArmorM-Llama3-8B are already lightweight models, the fine-tuning strategy of freezing their weights doesn't allow the different layers of neurons to learn the complexity of certain vulnerability patterns.

B. LLMs Adaptation Methods

Understanding architecture of LLMs requires a background knowledge of attention mechanisms[1] [2] and transformers [9]. The first Transformer architecture presented by Vaswani et al. in [9] is an **Encoder-Decoder** architecture capable of extracting meaning from a text sequence and understanding the relationships between words. Depending on the training strategies and the way in which the input data is processed, two other types of architecture have emerged in the literature: **Encoder-Only** models and **Decoder-Only** models.

Various methods are available to adapt LLMs to specific domain or tasks: Pre-training [10], Continued pre-training (second-stage pre-training) [11], Fine-tuning [12], Retrieval Augmented Generation (RAG) [13], In-context learning (ICL) [14].

Depending on the task, it's important to choose the right adaptation method to take advantage of the LLM's knowledge. Criteria for selection may be related to the LLM's ability to perform the task, domain specificity, availability of data and computational resources for training and inference. In our work, we will use fine-tuning as a method for adapting LLMs. Fine-tuning strategies are less demanding in terms of computing resources and time consumption than pre-training and continued pre-training. One of the best-known techniques is Parameter-Efficient Fine-Tuning (PEFT). The goal is to reduce the number of fine-tuning parameters and memory usage while maintaining performance similar to full fine-tuning [15]. By training a reduced number of parameters and preserving most of the LLM structure, PEFT saves time and computing resources [16]. The main fine-tuning techniques found in the literature are Supervised Fine-Tuning (SFT), Low-Rank Adaptation (LoRA) [17], Quantization [18], [19], Reinforcement Learning from Human Feedback (RLHF) [20], Knowledge Distillation, etc.

III. Methodology

To fully exploit the potential of LLMs for detecting web attacks, the methodology proposed in [3] is used. It consists of three stages: exploration, experimentation and evaluation (see Fig. 1). Three (03) LLMs have been used, taking into account the different types of architecture. These are BERT (Encoder-Only), Llama (Decoder-Only) and Flan-T5 (Encoder-Decoder).

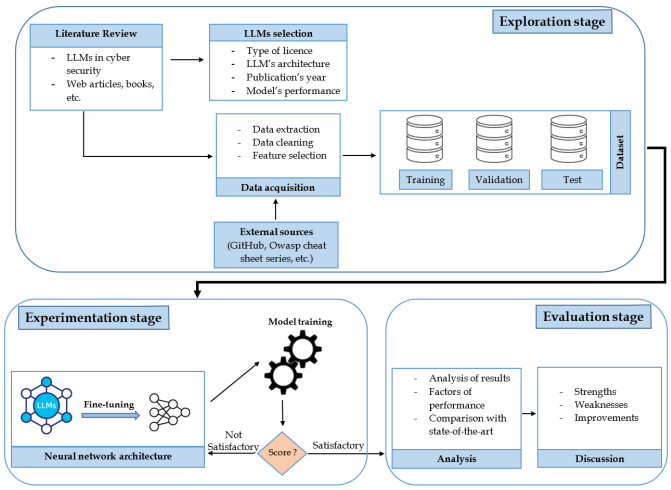

Fig. 1. Overview of methodological framework.

A. Dataset

We collected payloads from several sources[3] [4] [5] including [21], [22]. Theses payloads are processed and cleaned accord-

[1] https://medium.com/@geetkal67/attention-networks-a-simple-way-to-understand-self-attention-part2-d840189e0e59

[2] https://www.analyticsvidhya.com/blog/2024/01/different-types-of-attention-mechanisms/

[3] https://github.com/foospidy/payloads

[4] https://link.springer.com/chapter/10.1007/978-3-319-74690-6_20

[5] https://www.kaggle.com/datasets/syedsaqlainhussain/cross-site-scripting-xss-dataset-for-deep-learning

Authorized licensed use limited to: Nokia. Downloaded on September 18,2025 at 07:23:36 UTC from IEEE Xplore. Restrictions apply.

ing to our methodological framework presented in Fig. 1. Specifically, the pre-processing involved: (i) removing duplicate entries; (ii) normalizing scripts and delimiters to a standard format; (iii) filtering out payloads with corrupted or incomplete URLs; and (iv) truncating excessively long payloads beyond model input length. These steps ensured consistency and improved the learning during model fine-tuning. The final dataset obtained after pre-processing contains 141,961 payloads divided into five (05) categories:

- 82,316 benign payloads (normal traffic)
- 25,238 XSS payloads
- 22,163 Directory Traversal (or Path Traversal) payloads
- 11,735 SQL injection payloads
- 509 Command injection payloads

Table I gives an overview of our dataset.

TABLE I
DATASET OVERVIEW

Payloads	Label
http://www.wikihow.com/play-hunger-games-with-barbies&t=1396633468370&n=5529689&k=mainentity	benign
http://www.blogcu.com/destek/?q="><script> alert(document.cookie)</script>&bul=ara	xss
.%00.%f8%80%80%80%afetc%f8%80%80%80%afpasswd	dt
1' where 8864=8864;select count(*) from sysibm.systables as t1,sysibm.systables as t2,sysibm.systables as t3–	sqli
<!–#exec%20cmd="/bin/cat%20/etc/passwd"–>	cmdi

This dataset was used to fine-tune LLM models (BERT, Llama, Flan-T5) with the aim of specializing them in the detection and classification of attacks against web applications. This is a multi-class classification problem, where each input is classified into one of five (05) classes (Benign, CMDi, DT, SQLi, XSS). The model thus learns to recognize features of each class. This modeling is more representative of real-world environments. Indeed, when an attack occurs, the challenge for security administrators is not to know that there has been an attack, but to determine the nature of the attack in order to take corrective action.

B. Fine-tuning Implementation

a) Experimental setup: To implement fine-tuning techniques, we used Jupyter Notebook on OVH Cloud[6]. These Notebooks ran on an environment with the following characteristics:

- GPU: (1 × NVIDIA L4, 1 × 24 GiB RAM)
- CPU : 20 vCores
- RAM : 80 GiB
- Public network: 5 GB/s
- Framework : pytorch - pytorch2.4.0-py311-cuda11.8

The programming language used for scripting is Python, with a range of libraries including: transformers, pytorch, torch, accelerate, pandas, huggingface-hub, matplotlib, numpy, requests, safetensors, scikit-learn, seaborn, sentencepiece, tokenizers, torchaudio, torchtune, torchvision, etc.

Before submitting our dataset to the different models, we subdivided it into two parts: training dataset (80%) and test dataset (20%) with *train_test_split* function in *sklearn.model_selection*.

Our fine-tuning process for the BERT, Llama, and Flan-T5 models is shown in Fig. 2, 3, and 4, respectively. The base models used for each LLM are: *google-bert/bert-base-uncased, meta-llama/Llama-3.2-1B-Instruct, google/flan-t5-base*, respectively. These models were obtained from the Hugging Face [7] platform. We evaluated the performance of the base models by applying them to our test dataset. This allows us to assess the impact of fine-tuning on the performance of these models.

b) Fine-tuning Process for BERT: Supervised Fine-Tuning is used as fine-tuning technique for BERT model. Several scenarios have been experimented. These included freezing the weights of BERT model; then progressively adding layers of neural networks. During fine-tuning, only the additional layers are trained and their weights are updated. A final experiment consists of making trainable BERT model layers (full fine-tuning).

- **Scenario 1**: BERT base (frozen layers) + 1 classification layer
- **Scenario 2**: BERT base (frozen layers) + 1 fully connected layer + 1 classification layer
- **Scenario 3**: BERT base (frozen layers) + 2 fully connected layers + 1 classification layer
- **Scenario 4**: (full fine-tuning) BERT base + 2 fully connected layers + 1 classification layer

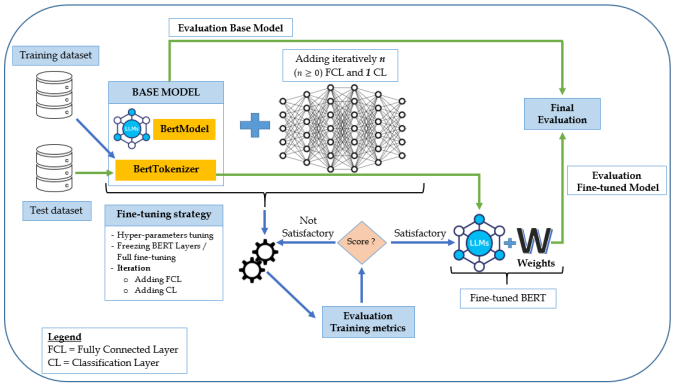

Fig. 2. Our Fine-tuning process for BERT.

c) Fine-tuning Process for Llama: QLoRA is used to fine-tune *Llama-3.2-1B-Instruct*. It combines the efficiency of LoRA with quantization techniques to further reduce memory usage and computational resources during fine-tuning. Llama-3.2-1B-Instruct parameters are quantized to 4-bit precision for storage, but computations (activations, gradients, optimizer

[6]https://www.ovhcloud.com/fr/

[7]https://huggingface.co/models

Authorized licensed use limited to: Nokia. Downloaded on September 18,2025 at 07:23:36 UTC from IEEE Xplore. Restrictions apply.

states) are performed in higher precision using bf16 to preserve accuracy. We used torchtune[8], a PyTorch library for implementation.

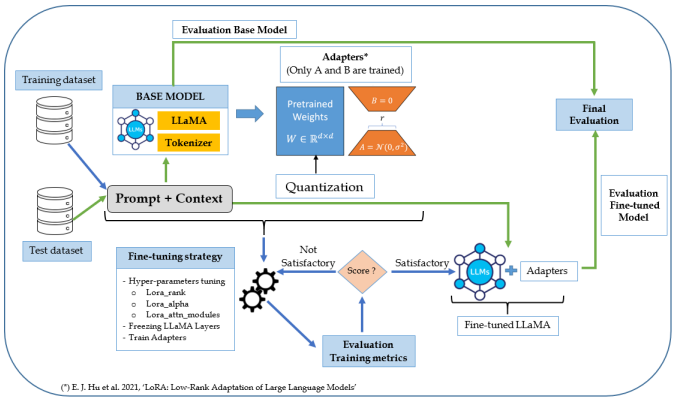

Fig. 3. Our Fine-tuning process for Llama.

d) Fine-tuning Process for Flan-T5: The fine-tuning technique used for the *flan-t5-base* model is supervised fine-tuning. We performed a full fine-tuning where all model parameters are trained.

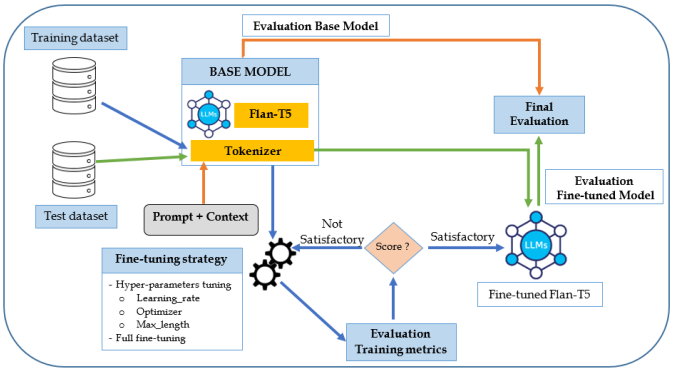

Fig. 4. Our Fine-tuning process for Flan-T5.

e) Evaluation metric: To evaluate this classification tasks, we used some common metrics like: *Confusion matrix*, *Precision* (Pr), *Recall* (Re), *F1-score* (F1), *Accuracy* (Acc).

In this work, we use unbalanced datasets (each class is not equally represented). To better evaluate the performance of our models, other metrics are proposed in the literature [23] which are more adapted for unbalanced datasets. These are: Balanced Accuracy (BA) [24], Geometric Mean (GM) [25], Matthews correlation coefficient (MCC) [26], Fowlkes-Mallows Index (FMI) [27]. Functions available in the scikit-learn[9] and imbalanced-learn[10] libraries were used to compute all metrics.

IV. Results

The table II shows the results of the evaluations of the base and fine-tuned models based on Scenario 1, Scenario 2, Scenario 3, and Scenario 4. For all metrics, the fine-tuned models outperformed the base model. Thus, fine-tuning improved the ability of the base model to detect and classify Benign, CMDi, DT, SQLi, and XSS payloads. We find that freezing the weights of the *bert-base-uncased* model and adding a classification layer (Scenario 1) yields moderate results: Accuracy (0.86), Balanced_Accuracy (0.63); contrary to the results of the Le et al. [8]. On the other hand, the addition of a fully connected layer (Scenario 2) resulted in performance gains for all metrics of the order of (+0.10) for Accuracy, (+0.12) for Balanced_Accuracy, (+0.17) for MCC, (+0.14) for FMI. This performance improvement continues with the addition of a second fully connected layer (Scenario 3), with good scores for MCC (0.95) and FMI (0.96). The model becomes increasingly robust to data imbalance. In Scenario 4, the weights of the BERT model and the additional layers are updated during the fine tuning. The results show that the model has learned to detect payload features with an accuracy rate of 0.99, BA(0.98), MCC(0.99), GM(0.97), FMI(0.99). The model obtained from Scenario 4 is therefore more resistant to unbalanced data. The model recognizes the least represented classes (CMDi payload). However, when the *bert-base-uncased* model weights are frozen, the least represented classes are not predicted or are predicted incorrectly (see Fig. 5). This is confirmed by the Geometric Mean obtained when evaluating *bert-base-uncased*, Scenario 1, Scenario 2, and Scenario 3. Indeed, if at least one class is unrecognized by the classifier, Geometric Mean resolves to zero.

TABLE II
Results of Fine-tuning Strategy of Bert Model

	bert-base-uncased	**Scenario 1**	**Scenario 2**	**Scenario 3**	**Scenario 4**
Acc	0.04	0.86	0.96	0.97	**0.99**
WAPr[a]	0.34	0.87	0.96	0.97	**1.00**
WARe[b]	0.04	0.86	0.96	0.97	**1.00**
WAF1[c]	0.07	0.86	0.96	0.97	**1.00**
BA	0.18	0.63	0.75	0.77	**0.98**
MCC	-0.01	0.76	0.93	0.95	**0.99**
GM	0.0	0.00	0.00	0.00	**0.97**
FMI	0.59	0.79	0.93	0.96	**0.99**

[a] WAPr : Weighted Average Precision.
[b] WARe : Weighted Average Recall.
[c] WAF1 : Weighted Average F1-score.

The table III compares the results of all models before and after fine-tuning. Evaluation of the *Llama-3.2-1B-instruct* model before fine-tuning yielded 346 predictions outside the 5 expected classes and 125 predictions outside the 5 expected classes for evaluation of the *flan-t5-base* model, despite writing prompts to clarify the context. After fine-tuning we see an improvement in the performance of each model for all the metrics used.

For the Llama model, we have an accuracy of 0.98, an improvement of (+0.55) over the base model (*Llama-3.2-1B-*

[8]https://pytorch.org/torchtune/stable/overview.html
[9]https://scikit-learn.org/stable/index.html
[10]https://imbalanced-learn.org/stable/index.html

Authorized licensed use limited to: Nokia. Downloaded on September 18,2025 at 07:23:36 UTC from IEEE Xplore. Restrictions apply.

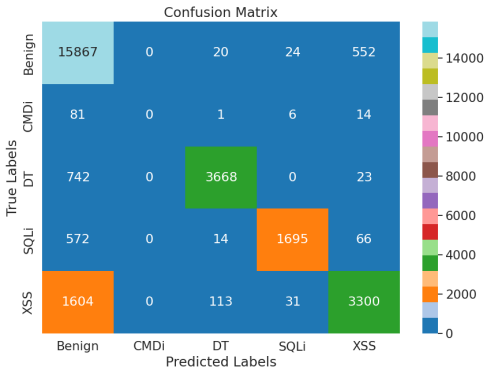

Fig. 5. Results of Scenario 1 : Confusion matrix.

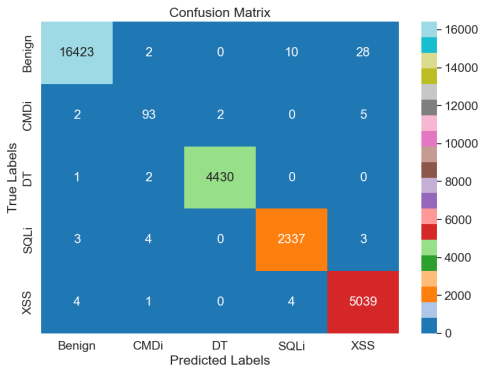

Fig. 6. Confusion matrix of BERT Fine-tuning.

TABLE III
RESULTS OF FINE-TUNING LLMS

	bert-base-uncased	**bert base + 2FCL + 1CL (full fine-tuning)**	**Llama-3.2-1B-instruct**	**Fine-tuned Llama**	**flan-t5-base**	**Fine-tuned flan-t5-base**
Acc	0.04	**0.99**	0.43	**0.98**	0.38	**0.99**
WAPr	0.34	**1.00**	0.46	**0.98**	0.58	**1.00**
WARe	0.04	**1.00**	0.44	**0.99**	0.39	**1.00**
WAF1	0.07	**1.00**	0.43	**0.98**	0.39	**1.00**
BA	0.18	**0.98**	0.30	0.79	0.30	**0.98**
MCC	-0.01	**0.99**	0.12	**0.97**	0.17	**0.99**
GM	0.0	**0.97**	0.09	0.54	0.09	**0.98**
FMI	0.59	**0.99**	0.45	**0.97**	0.48	**0.99**

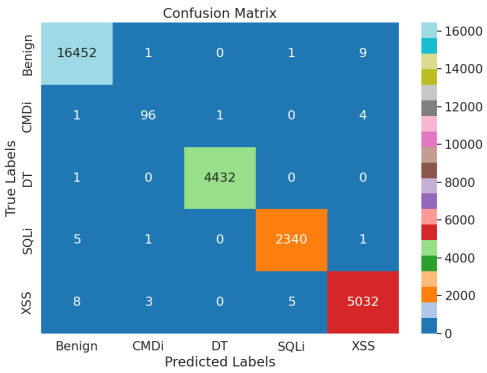

Fig. 7. Confusion matrix of Flan-T5 Fine-tuning.

instruct), BA(0.79) an improvement of (+0.49) over the base model. The fine-tuning technique (QLoRA) used improved the ability of the *Llama-3.2-1B-instruct* model to detect and classify payloads (Benign, CMDi, DT, SQLi, XSS).

For the Flan-T5 model, the accuracy rate was 0.99, an improvement of (+0.61) over the base model (*flan-t5-base*), BA(0.98) an improvement of (+0.68) over the base model. The fine-tuning technique (SFT) used improved the *flan-t5-base* model's ability to detect and classify payloads (Benign, CMDi, DT, SQLi, XSS). In addition to the improved performance, we note that the BERT and Flan-T5 models obtained after fine-tuning are more robust in the context of unbalanced dataset. In fact, for both models, we observe a similarity between the values of the classical metrics (accuracy, weighted average precision, weighted average recall, weighted average F1-score) and the metrics adapted to unbalanced dataset (BA, MCC, GM, FMI). This result is confirmed by the confusion matrix of these two models (see Fig. 6,7).

The least represented class (CMDi) registers 93 good predictions out of 102 for BERT and 96 good predictions out of 102 for Flan-T5. The Llama model, on the other hand, is more sensitive to unbalanced dataset. Llama's fine-tuned model made only 5 good predictions of the CMDi class out of a total of 102 (see Fig. 8). This explains the low Geometric Mean score (0.54) observed for this model and a lower Balanced Accuracy score (0.79) than the accuracy score (0.98). *Llama-3.2-1B-instruct* did not learn enough about the features of the CMDi class due to its low representativeness. On the other hand, for the other more or less well represented classes (DT, SQLi, XSS), all fine-tuned models (BERT, Llama, Flan-T5) showed good rate of precision, recall and F1-score (see table III).

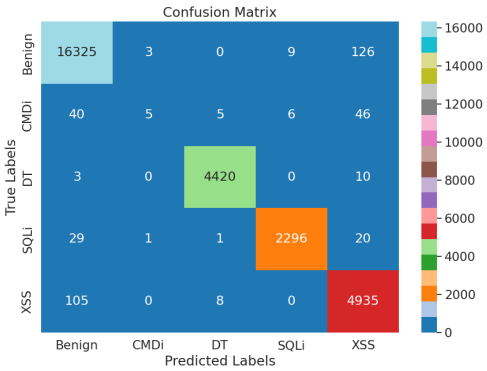

Fig. 8. Confusion matrix of Llama Fine-tuning.

Authorized licensed use limited to: Nokia. Downloaded on September 18,2025 at 07:23:36 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

In this article, we have presented the different types of LLM architecture and methods of LLMs adaptation. Based on a dataset collected from public sources, we fine-tuned *bert-base-uncased*, *Llama-3.2-1B-instruct*, *flan-t5-base* models for the detection and classification of web application attacks (XSS, DT, CMDi, SQLi) as well as benign payloads recognition. The main contributions of this study are as follows:

- Proposal and implementation of a fine-tuning process for the detection and classification of web attacks (CMDi, DT, SQLi, XSS), as well as for the detection of normal traffic, using the following models: *bert-base-uncased, Llama-3.2-1B-instruct* and *flan-t5-base*
- Confirmation that fine-tuning enables LLMs to specialise in performing cybersecurity tasks efficiently
- Demonstrating that freezing the base model weights during fine-tuning and adding a classification layer yields limited results. Conversely, fine-tuning performance improves when fully connected layers are added progressively, followed by a classification layer
- The impact of unbalanced data on the performance and robustness of fine-tuning is also examined. This should inform the selection of fine-tuning techniques and evaluation measures.

References

[1] Check Point Team, "Check point research reports highest increase of global cyber attacks seen in last two years – a 30% increase in q2 2024 global cyber attacks," Check Point, Jul. 2024. [Online]. Available: https://blog.checkpoint.com/research/check-point-research-reports-highest-increase-of-global-cyber-attacks-seen-in-last-two-years-a-30-increase-in-q2-2024-global-cyber-attacks/. Accessed 05 Jan 2025

[2] R. L. Alaoui and E. H. Nfaoui, "Deep learning for vulnerability and attack detection on web applications: a systematic literature review," Future Internet, vol. 14, no. 4, 2022, doi: 10.3390/fi14040118.

[3] S. R. Nana, D. Bassolé, D. Guel, and O. Sié, "Deep learning and web applications vulnerabilities detection: An approach based on large language models," International Journal of Advanced Computer Science and Applications, vol. 15, no. 7, 2024.

[4] G. Deng et al., 2024, "PENTESTGPT: Evaluating and harnessing large language models for automated penetration testing," In Proceedings of the 33rd USENIX Conference on Security Symposium (SEC '24). USENIX Association, USA, Article 48, 847–864.

[5] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 2082–2086. doi: 10.1145/3611643.3613083.

[6] M. Hassanin and N. Moustafa, "A comprehensive overview of large language models (LLMs) for cyber defences: opportunities and directions," May 23, 2024, arXiv: arXiv:2405.14487. [Online] Available: http://arxiv.org/abs/2405.14487

[7] D. Cao, Y. Liao, and X. Shang, "RealVul: Can We Detect Vulnerabilities in Web Applications with LLM?," in Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds., Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8268–8282. doi: 10.18653/v1/2024.emnlp-main.472.

[8] T. -T. -H. Le, A. A. Adiputra, Y. Hwang, J. Son and H. Kim, "Fine-tuning transformer LLMs for detecting SQL injection and XSS vulnerabilities," 2025 International Conference on Artificial Intelligence in Information and Communication (ICAIIC), Fukuoka, Japan, 2025, pp. 0946-0951, doi: 10.1109/ICAIIC64266.2025.10920868.

[9] A. Vaswani et al., "Attention is all you need," In Advances in Neural Information Processing Systems, Curran Associates, Inc., 2017.

[10] Rick Merritt, "What are foundation models?," NVIDIA, Feb. 2025. [Online]. Available: https://blogs.nvidia.com/blog/what-are-foundation-models/. Accessed 7 Mar 2025

[11] A. Jain, A. Maleki, and N. Saade, "Methods for adapting large language models," AI Meta, Aug. 2024. [Online]. Available: https://ai.meta.com/blog/adapting-large-language-models-llms/. Accessed 7 Mar 2025

[12] Dave Bergmann, "What is fine-tuning?," IBM, Mar. 2024. [Online]. Available: https://www.ibm.com/think/topics/fine-tuning. Accessed 7 Mar 2025

[13] Alexander S. Gillis, "What is retrieval-augmented generation (RAG) in AI?," TechTarget, Dec. 2024. [Online]. Available: https://www.techtarget.com/searchenterpriseai/definition/retrieval-augmented-generation. Accessed 7 Mar 2025

[14] Deval Shah, "What is in-context learning, and how does it work: the beginner's guide," LAKERA, Mar. 2025. [Online]. Available: https://www.lakera.ai/blog/what-is-in-context-learning. Accessed 29 Mar 2025

[15] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, "Parameter-efficient fine-tuning methods for pretrained language models: a critical review and assessment," Dec. 19, 2023, arXiv: arXiv:2312.12148. doi: 10.48550/arXiv.2312.12148.

[16] C. Stryker and I. Belcic, "What is parameter-efficient fine-tuning (PEFT)?", IBM, Mar. 2024. [Online]. Available: https://www.ibm.com/think/topics/parameter-efficient-fine-tuning. Accessed 05 Dec 2024

[17] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," Oct. 16, 2021, arXiv: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.

[18] Rupansh Parth Kaushik, "Fine-tuning LLMs using PEFT,", LearnOpenCV, Feb. 2024. [Online]. Available: https://learnopencv.com/fine-tuning-llms-using-peft/. Accessed 11 Dec 2024

[19] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLORA: efficient finetuning of quantized LLMs," in Proceedings of the 37th International Conference on Neural Information Processing Systems, in NIPS '23. Red Hook, NY, USA: Curran Associates Inc., Dec. 2023, pp. 10088–10115.

[20] H. Lang, F. Huang, and Y. Li, "Fine-tuning language models with reward learning on policy," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), K. Duh, H. Gomez, and S. Bethard, Eds., Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 1382–1392. doi: 10.18653/v1/2024.naacl-long.75.

[21] Y. Fang, C. Huang, Y. Xu, and Y. Li, "RLXSS: Optimizing XSS detection model to defend against adversarial attacks based on reinforcement learning," Future Internet, vol. 11, p. 177, Aug. 2019, doi: 10.3390/fi11080177.

[22] Morzeux - Available: https://github.com/Morzeux/HttpParamsDataset. Accessed 09 Dec 2024

[23] Z. Sawadogo, G. Mendy, J. M. Dembele, and S. Ouya, "Android malware detection: investigating the impact of imbalanced data-sets on the performance of machine learning models," 2022 24th International Conference on Advanced Communication Technology (ICACT), PyeongChang Kwangwoon_Do, Korea, Republic of, 2022, pp. 435-441, doi: 10.23919/ICACT53585.2022.9728833.

[24] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, "The balanced accuracy and its posterior distribution," 20th International Conference on Pattern Recognition, Istanbul, Turkey, 2010, pp. 3121-3124, doi: 10.1109/ICPR.2010.764.

[25] R. Barandela, J. S. Sánchez, V. Garcıa, and E. Rangel, "Strategies for learning in class imbalance problems", Pattern Recognition, 36(3), (2003), pp 849-851.

[26] S. Boughorbel, F. Jarray, and M. El-Anbari, "Optimal classifier for imbalanced data using Matthews Correlation Coefficient metric," PLoS ONE, vol. 12, no. 6, p. e0177678, Jun. 2017, doi: 10.1371/journal.pone.0177678.

[27] E. B. Fowlkes, and C. L. Mallows, "A method for comparing two hierarchical clusterings," Journal of the American Statistical Association, vol. 78, no. 383, pp. 553–569, 1983. [Online]. Available: https://www.tandfonline.com/doi/abs/10.1080/01621459.1983.10478008

Authorized licensed use limited to: Nokia. Downloaded on September 18,2025 at 07:23:36 UTC from IEEE Xplore. Restrictions apply.